HW4: AI Harness System Design and Analysis

1. Problem Definition and Application Background

This project designs an **AI Paper Reading and Presentation Assistant** for students who need to understand technical research papers and prepare oral presentations. The target user is a student in a technical course who uploads a research paper PDF and asks the system to produce a structured paper summary, concept explanations, a slide outline, a speaking script, and a presentation readiness evaluation.

The problem is not simply text generation. Research papers contain specialized terminology, mathematical notation, experimental details, citations, and discipline-specific assumptions. A student may understand some parts of a paper but struggle to connect the motivation, method, results, and limitations into a coherent presentation. A normal chatbot can provide useful help, but it may skip important sections, lose track of evidence, or generate claims that are not grounded in the paper.

This homework therefore treats the assistant as an **AI harness system** rather than a model-training task. The goal is to design the surrounding system that makes an LLM useful, controllable, and inspectable. The design focuses on how the LLM acts as a controller, how tools are called, how memory is organized, how the workflow is orchestrated, and how outputs are evaluated before they are presented to the student.

The current project should be understood as a **design and mock prototype**. It defines the architecture, tool boundaries, and orchestration logic, but it does not claim to be a fully deployed production system.

2. AI Harness System Design

The proposed system is built around an LLM controller that coordinates a sequence of specialized tools. The controller does not directly solve the entire task in one prompt. Instead, it decomposes the assignment into smaller steps: parse the paper, identify content, summarize sections, explain difficult concepts, generate presentation materials, and evaluate readiness.

LLM as Controller

The LLM controller receives the user request and decides which tool should be called next. It maintains the current task state, checks whether required information is missing, and decides whether the output is ready or needs revision. For example, if the paper text has not been parsed, the controller should call `parse_paper_pdf`. If a summary exists but the slide outline is missing, it should call `generate_slide_outline`. If the readiness evaluation is weak, the controller should revise the weakest component.

In this design, the LLM has three main responsibilities:

1. **Planning**: choose a sequence of tool calls based on the user's goal.
2. **Interpretation**: convert tool outputs into useful academic explanations.
3. **Quality control**: inspect intermediate outputs and request revisions when necessary.

Tools

The tool layer contains functions with explicit inputs and outputs. Some tools are deterministic, such as PDF parsing or formatting a slide outline. Other tools may use the LLM internally, such as summarization or concept explanation. The important design principle is that each tool has a narrow responsibility and can be tested separately.

The minimum tool set includes:

- `parse_paper_pdf`
- `summarize_section`
- `explain_concept`
- `generate_slide_outline`
- `evaluate_presentation_readiness`

Additional tools could later be added for citation checking, figure extraction, equation explanation, or slide deck export.

Memory

The memory layer stores information needed across the workflow. The design uses two types of memory:

- **Short-term task memory**: the current paper text, section map, section summaries, concept explanations, slide outline, speaking script, and evaluation feedback.
- **Long-term user memory**: the student's preferred explanation level, common presentation format, previous feedback, and recurring concepts from earlier papers.

Short-term memory supports the current paper-reading session. Long-term memory personalizes future sessions without retraining the model.

Data Flow

The basic data flow is:

1. The student uploads a PDF and provides presentation preferences.
2. The controller calls `parse_paper_pdf` to extract text and metadata.
3. The controller calls summarization and explanation tools on selected sections.
4. The controller calls presentation-generation tools to create an outline and script.
5. The controller calls the evaluator to score the output.

6. If the score is below the target threshold, the controller revises the weak component.
7. The final output is returned to the student with a readiness score and revision suggestions.

This flow makes the system easier to inspect because intermediate artifacts are preserved rather than hidden inside one long prompt response.

3. Tool Design

The following table defines the core tools, their inputs, outputs, and when they should be called.

Tool	Input	Output	When Called
`parse_paper_pdf`	PDF file path or uploaded PDF object	Raw text, title, authors, abstract, detected sections, references if available	First step after the user uploads a paper
`summarize_section`	Section name, section text, desired detail level	Section summary with key claims, methods, evidence, and limitations	After the paper is parsed and sections are identified
`explain_concept`	Concept name, local paper context, audience level	Plain-language explanation, analogy if useful, and relation to the paper	When the controller detects difficult terms or the user asks for clarification
`generate_slide_outline`	Structured paper summary, talk length, audience level	Slide-by-slide outline with title, goal, key points, and estimated timing	After summaries and concepts are available
`evaluate_presentation_readiness`	Summary, concept explanations, slide outline, speaking script, paper evidence	Score, rubric feedback, missing content, and revision recommendations	After presentation materials are generated
`generate_speaking_script`	Slide outline, audience level, desired speaking style	Speaker notes for each slide	After the slide outline is accepted or generated
`extract_key_terms`	Paper sections or full text	List of important concepts, acronyms, methods, datasets, and metrics	Before concept explanation or summary revision

The five required tools are the core of the system. The additional tools show how the design could be extended while keeping responsibilities separated.

Example Tool Contracts

`parse_paper_pdf(pdf_file)` should return structured content:

```text
{
  "title": "...",
  "authors": ["..."],
  "sections": {
    "abstract": "...",
    "introduction": "...",
    "method": "...",
    "experiments": "...",
    "conclusion": "..."
  }
}
```

`evaluate_presentation_readiness(outputs)` should return both a numeric score and actionable feedback:

```text
{
  "score": 82,
  "weaknesses": ["Experiment results are not explained clearly."],
  "recommendations": ["Revise the results slide with metrics from the paper."]
}
```

These contracts allow the orchestrator to reason about missing information and decide the next step.

4. Function Calling / Tool Usage Mechanism

The LLM controller uses function calling to interact with tools. Instead of asking the LLM to directly produce every output, the system gives it a list of available functions, each with a schema. The controller selects a function, provides arguments, receives a structured result, stores that result in memory, and decides the next action.

A simplified function-calling cycle is:

1. **Observe state**: check which artifacts already exist.
2. **Select tool**: choose the function needed for the next missing artifact.
3. **Call tool**: pass validated arguments.
4. **Store result**: save the tool output in short-term memory.
5. **Reflect**: determine whether the result is sufficient.
6. **Continue or revise**: proceed to the next tool or repair the current output.

For example, after `parse_paper_pdf` returns a section map, the controller may call `summarize_section` once for the abstract, once for the method section, and once for the experiments section. If the experiments section is missing, the controller should not invent results. It should mark the section as unavailable and ask the user for a clearer PDF or continue with a warning.

This mechanism reduces hallucination risk because the LLM must work through explicit tool outputs. It also improves debugging because every major step produces an inspectable artifact.

5. Agent Workflow

The agent workflow is a multi-step process:

1. **User input**: the student uploads a paper PDF and specifies audience level, presentation length, and output goals.
2. **Paper parsing**: the controller calls `parse_paper_pdf` to extract text, metadata, and section boundaries.
3. **Section summarization**: the controller calls `summarize_section` for major paper sections.
4. **Concept detection and explanation**: the controller identifies difficult concepts and calls `explain_concept` for each important concept.
5. **Slide planning**: the controller calls `generate_slide_outline` using the structured summary and presentation constraints.
6. **Script drafting**: the controller generates speaking notes for each slide.
7. **Readiness evaluation**: the controller calls `evaluate_presentation_readiness`.
8. **Revision loop**: if the score is below the threshold, the controller revises the weakest component.
9. **Final output**: the system returns the summary, explanations, slide outline, script, score, and improvement suggestions.

The workflow is mostly sequential, but it includes conditional branches. For instance, if the student asks only for concept explanations, the system may parse the paper and explain terms without generating slides. If the readiness score is low because the slides lack experimental evidence, the controller should return to the relevant section summaries and revise the slide outline.

6. AI Orchestration and Decision Logic

The orchestrator manages state and enforces the workflow. Its role is to keep the LLM controller grounded in the current task rather than allowing it to jump directly to a final answer.

A simplified decision policy is:

- If no paper text exists, call `parse_paper_pdf`.
- If paper sections exist but section summaries are missing, call `summarize_section`.
- If key terms exist but explanations are missing, call `explain_concept`.
- If summaries exist but no presentation outline exists, call `generate_slide_outline`.
- If the outline exists but no script exists, generate the speaking script.
- If all presentation materials exist, call `evaluate_presentation_readiness`.
- If readiness is below the target threshold, revise the weakest artifact and evaluate again.

The controller should also apply safety checks. It should avoid fabricating missing paper details, label uncertain outputs, and preserve links between generated claims and paper sections. For example, if the parsed paper does not contain a clear results section, the assistant should say that the results could not be confidently extracted rather than creating unsupported results.

This orchestration logic supports transparency. A student or instructor can inspect what happened at each stage: which tools were called, what data was produced, what decisions were made, and why a revision was requested.

7. Evaluation Method

The evaluation component checks whether the generated presentation package is useful, accurate, and ready for delivery. The readiness score is not meant to be an absolute truth. It is a structured estimate that helps guide revision.

Metric	What It Checks	Example Evaluation Question
Factual Accuracy	Whether claims are supported by the paper text	Are the stated contributions and results grounded in the paper?
Coverage	Whether major sections are represented	Does the summary include problem, method, experiments, results, and limitations?
Concept Clarity	Whether difficult ideas are explained at the right level	Would an undergraduate audience understand the core method?
Slide Coherence	Whether the slide outline has a logical flow	Does the presentation move from motivation to method to evidence to takeaway?
Timing and Delivery	Whether the script fits the requested talk length	Can the student present this script in the available time?
Readiness for Revision	Whether feedback is specific and actionable	Does the evaluator identify exactly what should be improved?

The evaluator returns:

- A score from 0 to 100
- A list of strengths
- A list of weaknesses
- Specific revision suggestions
- Optional warnings about missing or uncertain paper content

For example, a presentation might receive a high clarity score but a low coverage score if it explains the method well while ignoring limitations. The controller would then revise the slide outline to include a limitations or discussion slide.

8. Limitations and Future Improvements

The current design has several limitations. First, PDF parsing can be unreliable, especially for papers with multi-column layouts, equations, tables, or scanned pages. Second, LLM-generated summaries may still contain unsupported claims if grounding checks are weak. Third, presentation quality is partly subjective, so the readiness score should be treated as guidance rather than a final grade. Fourth, the current prototype files only define lightweight mock logic and do not implement full LLM API calls, vector memory, or real slide deck generation.

Future improvements could include:

- A stronger PDF parser with table, figure, and equation extraction.
- Citation grounding that links each generated claim to a paper section.
- Vector memory for retrieving relevant paper chunks during summarization.
- User-adjustable presentation style, such as formal, beginner-friendly, or conference-style.
- Export to PowerPoint or Google Slides.
- Human-in-the-loop revision where the student can approve or reject generated slides.
- A benchmark set of papers and rubrics for evaluating output quality.

These improvements would make the harness more robust while preserving the same basic architecture.

9. Conclusion

The AI Paper Reading and Presentation Assistant demonstrates how an LLM can be used as a system controller rather than only as a text generator. The system design separates responsibilities across tools, memory, orchestration, and evaluation. This separation makes the assistant more transparent, easier to debug, and better suited to multi-step academic work.

The main contribution of this homework design is the AI harness architecture: a controlled workflow that transforms a paper PDF into structured learning and presentation materials. The project does not depend on training a new model. Instead, it shows how careful system design can make existing AI capabilities more reliable and useful for students preparing technical presentations.